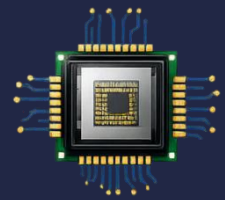


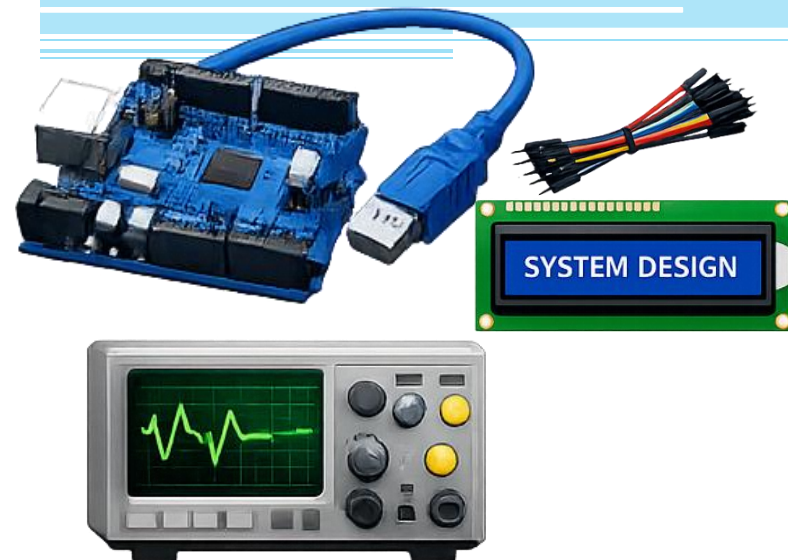
CS-301

Microprocessor- Based System Design



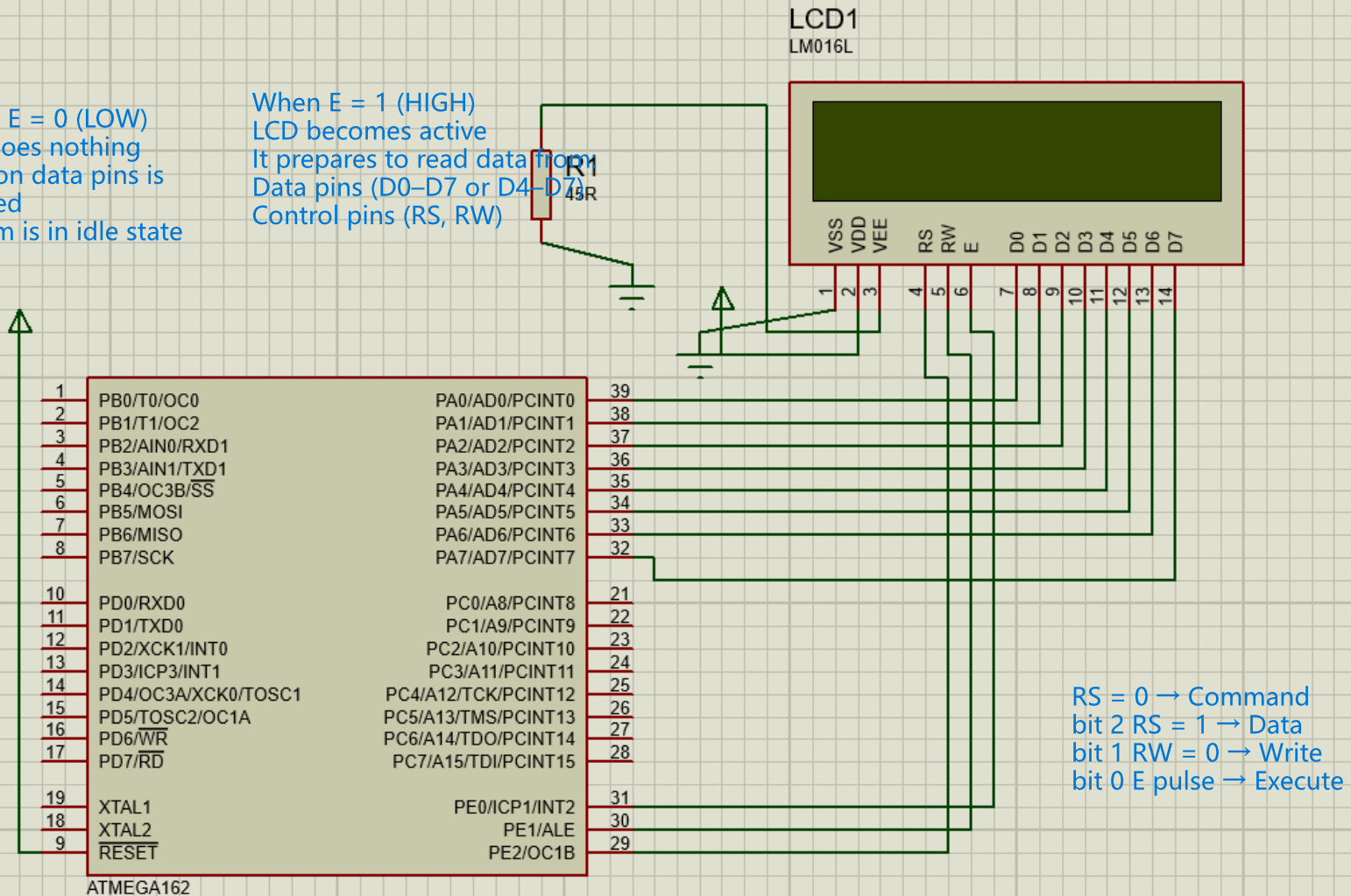
Lecture 15: Embedded C Code for Interfacing with LCDs and Keypads and Serial Communication Basic Concepts

Course Teacher: Syeda Ramish Fatima



When E = 0 (LOW)
LCD does nothing
Data on data pins is
ignored
System is in idle state

When E = 1 (HIGH)
LCD becomes active
It prepares to read data from
Data pins (D0–D7 or D4–D7)
Control pins (RS, RW)



Embedded C Code for LCD

LCD.c

```
#define F_CPU 1000000UL
#include<avr/io.h>
#include<util/delay.h>

void _LCDClear(void);
void _LCDFunctSet(void);
void _LCD_DispCtrl(void);
void _LCDEntryMod(void);
void _PrintChar(char k);

int main(void)
{
    DDRA=0xFF;
    DDRE=0xFF;
```

both set to output

Embedded C Code for LCD

LCD.c

```
while(1)
{
    _LCDClear();
    _LCDFunctSet();
    _LCD_DispCtrl();
    _LCDEntryMod();
    char k='a';
    for(int i=0; i<5;i++)
        { _PrintChar(k);
          _delay_ms(2000);
          k++;
        }
}

return 0;

} //main ends
```

Embedded C Code for LCD

LCD.c

```
void _LCDClear(void)
{
    porta = command;
    0x38 → Function Set: 8-bit mode, 2 lines, 5x8 dots
    per character
    0x3B → Function Set: same as above, but
    sometimes with cursor or display options enabled
    his line tells the LCD:
    "I want you to work in 8-bit mode and display 2
    lines of text with normal font.

    Wait 10 milliseconds so the LCD is ready to receive the command.

    void _LCDFunctSet(void)
    {
        this func configures lcd for 2 lines 8 bit mode

        PORTA = 0x01;
        PORTE = 0x00;
        _delay_ms(10);
        PORTE = 0x01;
        _delay_ms(5);
        PORTE = 0x00;
```

PORTA = 0x01;
is a command telling a clear command incoming

RS = 0 → command
RW = 0 → write
E = 0 → idle

Command 0x01 goes to LCD via PORTA
Control signals set via PORTE
Enable last bit 0 pulse tells LCD → "execute this"
LCD clears screen

```
        PORTA = 0x3B; // or it can be 0x38
        PORTE = 0x00;
        _delay_ms(10);
        PORTE = 0x01;
        _delay_ms(5);
        PORTE = 0x00;
```

Read the command I just put on the data pins."
Wait 5 milliseconds to ensure the LCD has time to read
the command.
Set EN back to 0 to complete the enable pulse.

```
}
```

Embedded C Code for LCD

LCD.c

```
void _LCD_DispCtrl(void)
{
    PORTA = 0x0F;  command : Display ON, Cursor ON, Cursor Blinking ON
    PORTE = 0x00;
    _delay_ms(10);
    PORTE = 0x01;
    _delay_ms(5);
    PORTE = 0x00;
}

void _LCDEntryMod(void)
{
    PORTA = 0x06;  After you print a character, move the cursor to the right
    PORTE = 0x00;
    _delay_ms(10);
    PORTE = 0x01;
    _delay_ms(5);
    PORTE = 0x00;
}
```

Embedded C Code for LCD

LCD.c

```
void _PrintChar(char K)
```

```
{
```

```
    PORTA = K;    character to display is K
```

```
    PORTE = 0x0C;
```

Give the LCD
5ms to process
the enable pulse.

```
    _delay_ms(5);
```

```
    PORTE = 0x0D;
```

```
    _delay_ms(10);
```

```
    PORTE = 0x0C;
```

0x0D → binary 00001101 → EN=1,RW=0 RS=1,

```
}
```

Wait 10ms for the LCD to finish writing the character to DDRAM.

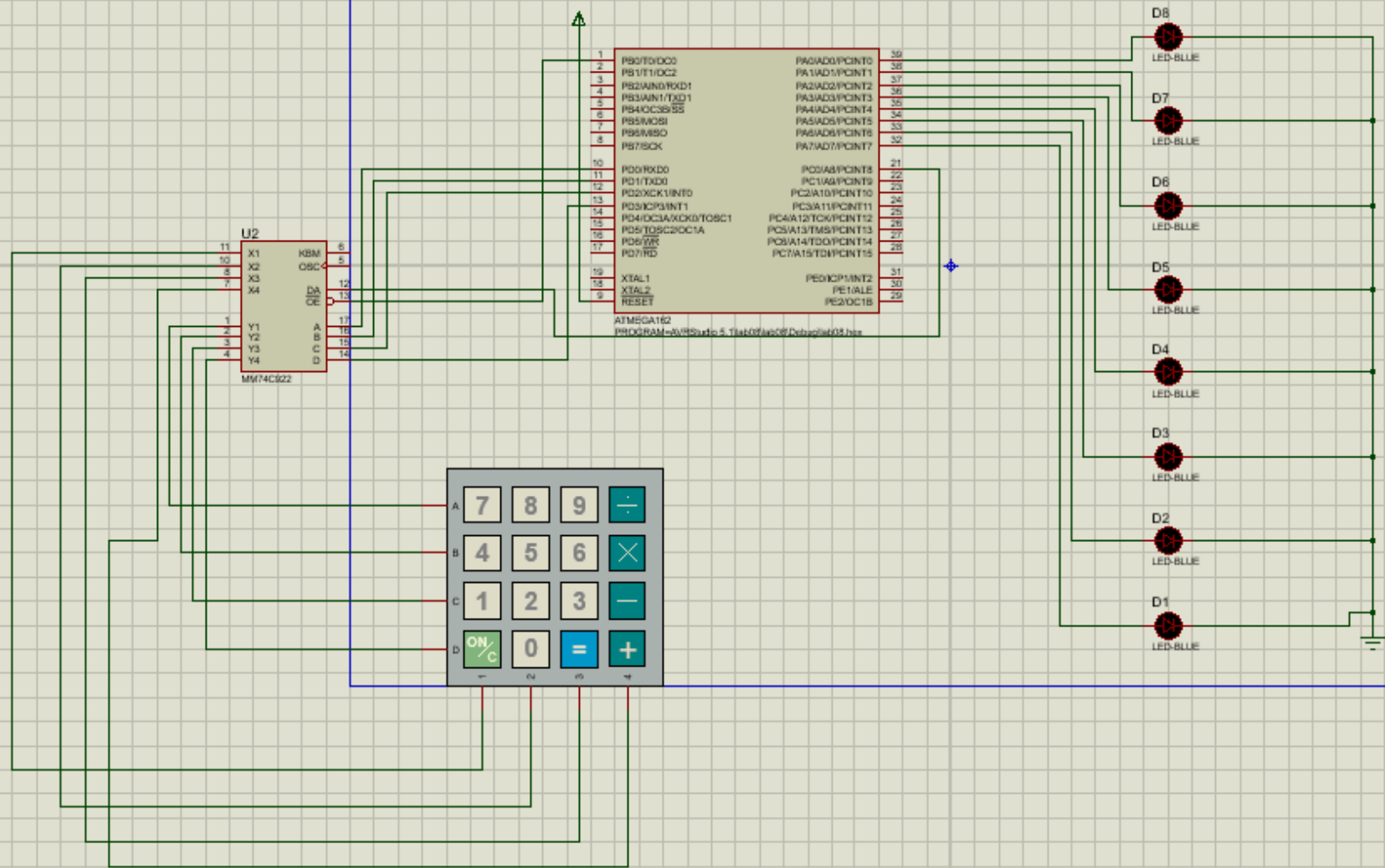
LCD latches the character, moves the cursor (as per Entry Mode 0x06), and gets ready for the next character.

porte = 00001100

RS = 1 → Data mode (character)

RW = 0 → Write

E = 0 → Idle



Embedded C Code for Keypad

```
#include<avr/io.h>
#include<inttypes.h>
#define F_CPU 1000000UL
#include<util/delay.h>
int GetKeyPressed(void);

int main(void)
{
    char
    digit[20]={ '1' , '2' , '3' , 'A' , '4' , '5' , '6' , 'B' , '7' , '8' , '9' , '
    C' , '*' , '0' , '#' , 'D' , '\0' };
        DDRD=0xF0; Upper 4 bits of PORTD as output (rows), lower 4 bits as input (columns)
        DDRA=0xFF; //Port A is set as output connected to
LED bank
        DDRC=0x00; c0 is input connected with pin12 the dav(pin12 signals mcu data is available read it)
        DDRB= (1<<PB0) ; // PORTB0 as output, connected to pin 13, PB0 is driving OE or controlling
initialize led, all digits = 0 PORTA=digit[16] This allows the MCU to activate or deactivate the encoder outputs.

        int key; //return value of the key pressed 0xFF
        for no key pressed
```

Embedded C Code for Keypad

```
while(1)
{
    key=GetKeyPressed();    call key pad scanning function
    if(key!=16)    if a key is pressed
        PORTA=digit[key]; show corresponding character
}

}

int GetKeyPressed(void)
{
    char x;
    PORTB=0x00;    clean port b, to read
    int data=16;    deefault, no key presesed
    x=PINC;        // Key Pressed Data Available
    if (x == 0x01)    if dav was one and the data is available
    {
        data=(PIND & 0x0F);    read the lower bit values of portD
        return data;
    }
    return 16;
}
```